# Layers as lenses: how a two-hidden-layer ReLU network learns an oblique decision tree, step by step

A handout for new collaborators

Harish G. and collaborators (`layer-lenses` repository, October 2026)

**Abstract**

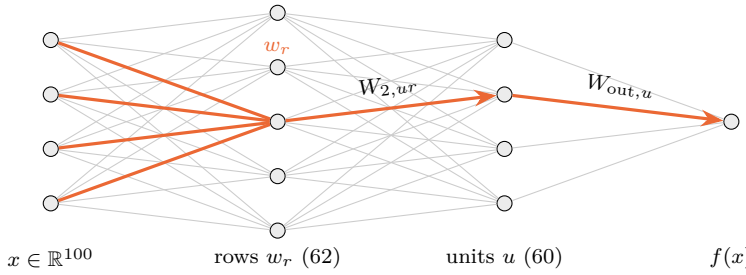
We study how gradient descent builds a hierarchy of features. The task is a depth-5 oblique decision tree (ODT) on the unit sphere in $\mathbb{R}^{100}$: 15 internal nodes with orthonormal normals route each input to one of 16 label nodes, where the label is the sign of a sixteenth normal. The network is a 62–60–1 ReLU network without biases. The central idea is that every first-layer row sees the data *through a lens*: the active region of the units that read it. Its gradient is the gradient of a linear classifier on data reweighted by that lens, so a row learns whatever direction the lens makes informative. We design an initialisation in which no row contains any internal normal and all the information about the tree sits in the wiring of the second layer, and we show that plain SGD then discovers the 15 internal normals level by level, builds 32 leaf detectors, and reaches 0.4 % test error. A symmetrised gradient turns the same runs into a simulation of population gradient flow, which separates the deterministic mechanism from symmetry breaking by sampling noise. This handout gives the definitions, the design, the learned representation, and the main trajectories, all from saved runs in the repository.

## 1 The output of a ReLU network as a sum over paths

Consider a network with two hidden layers and no biases, $f(x) = W_{\text{out}}\,\mathrm{ReLU}\big(W_2\,\mathrm{ReLU}(W_1 x)\big)$, with $W_1 \in \mathbb{R}^{62\times 100}$, $W_2 \in \mathbb{R}^{60\times 62}$, $W_{\text{out}} \in \mathbb{R}^{1\times 60}$. Write $w_r$ for row $r$ of $W_1$ and let $g_r(x) = \mathbf{1}[w_r \cdot x > 0]$ and $g_u(x) = \mathbf{1}[(W_2\,\mathrm{ReLU}(W_1 x))_u > 0]$ be the gates. Because $\mathrm{ReLU}(z) = g(z)\,z$,

$$f(x) \;=\; \sum_{u=1}^{60}\sum_{r=1}^{62} W_{\text{out},u}\, g_u(x)\, W_{2,ur}\, g_r(x)\,(w_r\cdot x) \;=\; \sum_{(u,r)} \underbrace{W_{\text{out},u}W_{2,ur}}_{\text{path weight}}\;\underbrace{g_u(x)g_r(x)}_{\text{path gate}}\,(w_r\cdot x). \tag{1}$$

The output is a sum over *paths* $(u, r)$; each path is a linear function of $x$ switched on inside a region of input space, the intersection of the active regions of its two units. The function is piecewise linear and positively homogeneous of degree 3 in the parameters.




path $(u,r)$ contributes $W_{\text{out},u}W_{2,ur}\,g_u(x)g_r(x)\,(w_r\cdot x)$

row $w_r$ sees the data through the lens $\lambda_r(x) = \sum_u W_{\text{out},u}W_{2,ur}\,g_u(x)g_r(x)$

Figure 1: The path decomposition (1). A row's gradient is a linear-classifier gradient on data reweighted by the lens $\lambda_r$.

**Gradient of a row.** For the logistic loss $\ell(y, f) = \log(1 + e^{-yf})$ the population gradient with respect to $w_r$ is

$$-\nabla_{w_r}\,\mathbb{E}\,\ell \;=\; \mathbb{E}\big[\sigma(-yf(x))\,y\,\lambda_r(x)\,x\big], \qquad \lambda_r(x) = \sum_u W_{\text{out},u}W_{2,ur}\,g_u(x)\,g_r(x). \tag{2}$$

This is the gradient of a *linear* classifier $w_r \cdot x$ on the data reweighted by $\sigma(-yf)\lambda_r(x)$. The weight $\lambda_r$ is what we call the row's lens: it is set by the units that read the row (their readout, their weight on the row, and their active region) and by the row's own gate. At initialisation $f \approx 0$ and $\sigma(-yf) \approx \frac{1}{2}$, so a row moves along $\sum_u W_{\text{out},u}W_{2,ur}\,\mathbb{E}[g_u g_r\,y\,x]$. Everything in this handout follows from asking what $\mathbb{E}[g_u g_r\,y\,x]$ is for the ODT data and for the units we build.

## 2 The data: a depth-5 oblique decision tree

Inputs are uniform on the unit sphere in $\mathbb{R}^{100}$. A fixed set of 31 orthonormal vectors $u_0, \dots, u_{30}$ (drawn once, tree seed 5178) is arranged as a complete binary tree in heap order (Figure 2): node $j$ has children $2j+1$ and $2j+2$. An input descends from the root: at internal node $j$ it goes to $2j+1$ if $u_j \cdot x < 0$ and to $2j+2$ otherwise. After four decisions it reaches a label node $m \in \{15, \dots, 30\}$ and the label is $y = \mathrm{sign}(u_m \cdot x)$. We call the 15 internal normals *routing normals* and the 16 others *leaf normals*; the region of inputs that reaches label node $m$ is the *leaf cell* of $m$, each of probability $1/16$. The label is noiseless, so the Bayes error is 0, and the task is invariant under every tree flip (negate $u_j$ and exchange the two subtrees below $j$), under rotations of the 69-dimensional orthogonal complement of the normals, and anti-invariant under negating all 16 leaf normals (which negates $y$). All runs use $10^7$ training points per epoch in minibatches of $10^4$ (1000 steps per epoch) and a fixed test set of $2 \times 10^5$ points.

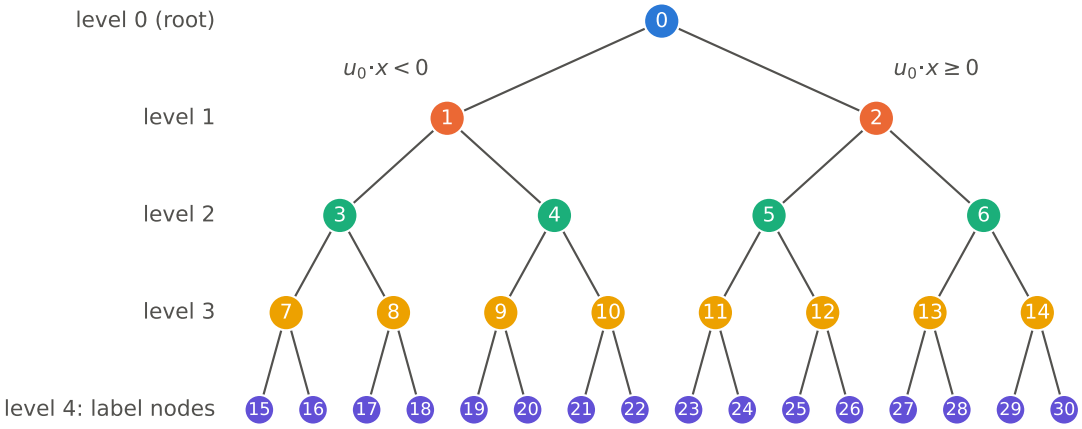



Figure 2: The tree. Internal nodes 0–14 route with the routing normals (colours mark the levels used throughout: level 0, level 1, level 2, level 3); label nodes 15–30 (violet) carry the leaf normals that set the label.

**What a linear model sees.** The best linear classifier is a useful reference. By strict convexity and the symmetries above, its population optimum is exactly $c\,v$ with $v = \sum_{m=15}^{30} u_m$, the sum of the leaf normals: every routing component and every off-span component is forced to zero. Numerically $c \approx 1.05$, the loss is 0.672 (against $\log 2 = 0.693$) and the error is 41.9 %. The direction $v$ matters below: it is the direction a row converges to when its lens is uninformative, and the designed cues are built around it.

## 3 The network and the designed initialisation

**Architecture.** 62 first-layer rows, 60 second-layer units, one output, no biases, plain SGD on the logistic loss with learning rate 0.1 and a cosine schedule to 0.005 over the run. The rows have two roles fixed at initialisation:

- **30 probes**, one per (internal node $j$, pole $\pm$). Each probe is $\sqrt{2}$ times a unit vector in the orthogonal complement of all 31 normals, the 30 directions being mutually orthogonal. A probe therefore starts with *no component on any tree normal*: it is pure noise with respect to the task.
- **32 cues**, one per (label node $m$, pole $\pm$): $\pm\sqrt{2}\,(v + \beta u_m)/\|v + \beta u_m\|$ with $\beta = 1$. A cue is the linear-model direction $v$ with a bias towards its own leaf normal; its cosine with $u_m$ is 0.46 and with every other leaf normal 0.23.

The second layer has four units per internal node $j$ (60 units for 15 nodes), with a block sign $b$ and a path sign $s \in \{\pm 1\}$, the four types $(b, s) \in \{(+,+), (+,-), (-,-), (-,+)\}$. Each unit:

- **vetoes one probe of its node**: weight $-1$ on the probe of pole $\mathrm{sign}(bs)$. With ReLU this cannot do anything until the probe carries a direction; it is where the probe's gradient enters.

- **reads the cues with a side contrast** (run 60's design, Figure 3): weight $s \cdot \text{pole} \cdot c_l$ on the cues of the leaves under $j$ on one side (the "open" side) and $-s \cdot \text{pole} \cdot c_l$ on the other side, nothing outside the subtree, with $c_l = 2^{-(5-l)/2}$ for a node of level $l$ $(0.18, 0.25, 0.35, 0.5)$.
- has readout $s \times 0.8165$. The readout's learning rate is multiplied by 80 (this replaces a third layer used in earlier runs).

Run 70 uses the same rows but "majority" cue weights (0.2 on the open side, 0.1 on the vetoed side, 0.1 outside, all times $s \cdot \text{pole}$), an extra $+1$ link to the partner probe of the vetoed one, and no readout speed-up. The whole family is parametrised by thirteen scalars $\alpha_1, \ldots, \alpha_{13}$ (row norms, cue bias, the three cue weights, the two probe links, the two readout sizes, two learning-rate scales, and the cues' in-span fraction) in `layer_lenses.alpha_init`.

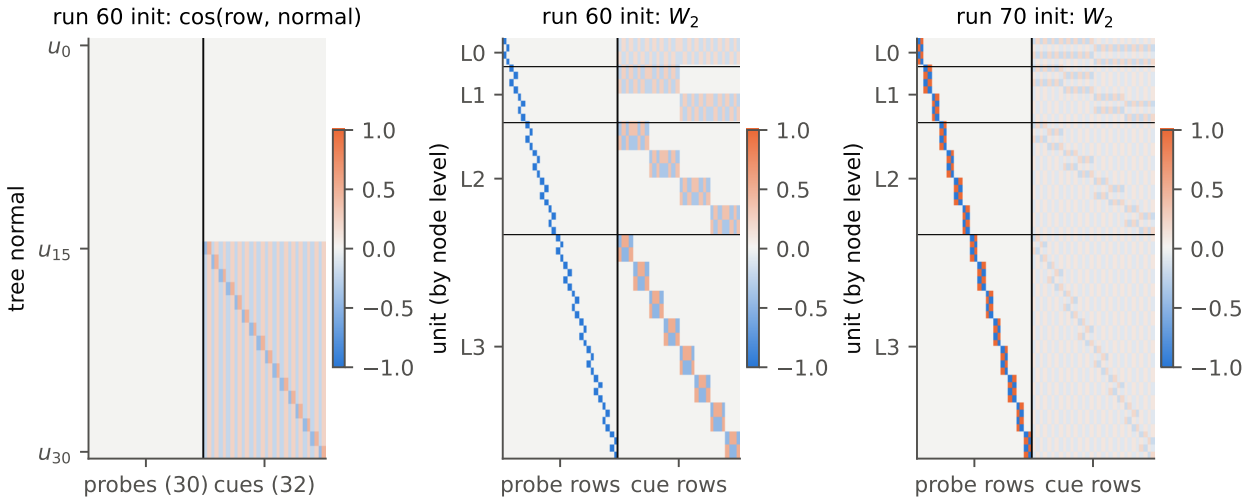



Figure 3: The initialisation. Left: cosines of the 62 rows of $W_1$ with the 31 normals in run 60. The probes have none; each cue has 0.46 on its own leaf normal and $\pm 0.23$ on the other fifteen (the sign is the cue's pole). Middle: $W_2$ of run 60, units grouped by the level of their node. The diagonal of $-1$ entries on the probe rows is the vetoes; the block pattern on the cue rows is the side contrast, with magnitude growing with the level. Right: $W_2$ of run 70 (majority weights, partner links).

**Why these initialisations are interesting.** Nothing in the first layer knows the 15 routing normals: the probes are random off-span directions, and the cues only know their own leaf. The tree is encoded in *which cues each unit reads and with which sign*. By (2), the probe vetoed by unit $u$ receives the push $W_{\text{out},u} W_{2,up}\, \mathbb{E}[g_u\, y\, x]$ once it has any direction at all, and $\mathbb{E}[g_u\, y\, x]$ restricted to a leaf cell $m$ has a component along every routing normal $a$ above $m$, with the sign of the side of $a$ on which $m$ lies. A unit whose cue weights favour the leaves on one side of node $j$ has an active region biased to that side, so the push on its probe along $u_j$ is $\propto \sum_{m \text{ under } j} \text{side}_j(m)\, p_u(m)$, where $p_u(m)$ is the probability that the unit is active on cell $m$. The side contrast of the design makes this sum large; a unit that reads both sides equally pushes nothing along $u_j$ (this is the failure of run 68, not shown). The same formula gives a push along every ancestor of $j$, which is why the probes acquire a transient ancestor component (Figure 4, middle). The init is also an exact fixed point of the tree symmetries, which is what makes Section 6 possible, and it is small enough that every quantity in the trajectory can be read off.

# 4 Results of runs 60 and 70

Both runs train in two phases: the cue rows are frozen for the first 500 epochs (so the probes learn with a fixed lens), then everything is free for 500 more. Table 1 and Figures 4–5 summarise them.

| run | init, optimiser | probe capture epoch, by level L0 | L1 | L2 | L3 | cues rotated by | loss / error / nodes at the end |
|---|---|---|---|---|---|---|---|
| 60 | run 60 design, SGD | 10 | 40 | 22 | 31 | 653 | 0.019 / 0.40 % / 31 of 31 |
| 70 | run 70 design, SGD | 99 | – | – | – | – | 0.140 / 4.0 % / 19 of 31 |
| 74 | run 60 design, symmetrised | 9 | 15 | 21 | 30 | 801 | 0.029 / 0.45 % / 31 of 31 |
| 114 | as 74, cosine restarted at epoch 500 | 10 | 39 | 22 | 30 | 598 | 0.006 / 0.05 % / 31 of 31 |
| 73 | run 70 design, symmetrised, 1000 frozen + 500 | 98 | 408 | 313 | 396 | – | 0.111 / 2.3 % / 15 of 31 |
| 97 | run 73 continued 2000 epochs, fresh cosine | (already captured) | | | | 1523 | 0.010 / 0.11 % / 31 of 31 |

Table 1: Capture = first epoch at which every probe of the level has $|\cos| > 0.95$ with its own routing normal; "nodes" counts the normals covered by some row at the end. Run 97's epochs are counted from the start of run 73.

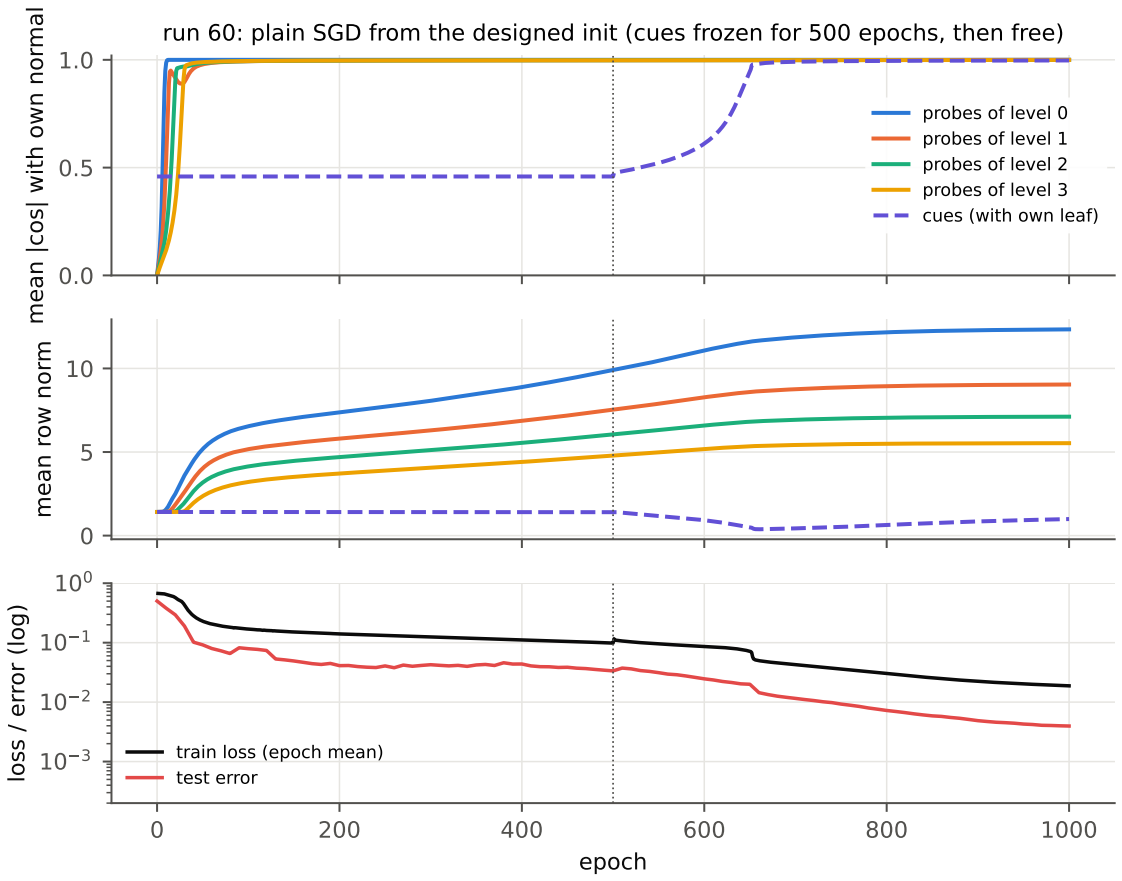



Figure 4: Run 60. Top: mean $|\cos|$ of the probes with their own routing normal, by level, and of the cues with their own leaf normal. The probes are captured within 10–40 epochs, root first; the cues (frozen until epoch 500, dotted line) rotate from 0.46 to 1 during the free phase. Middle: row norms grow level by level (root largest). Bottom: training loss and test error; the drop at epoch 650 is the cue rotation.

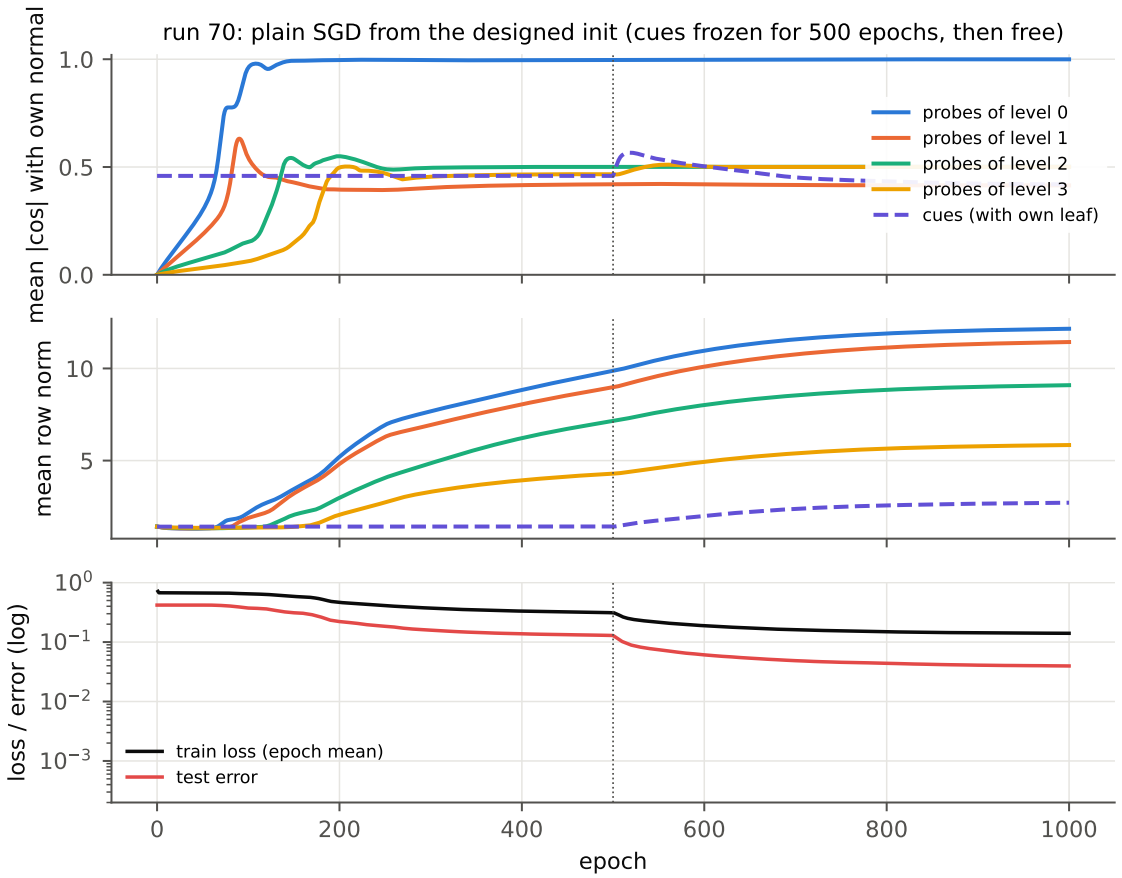



Figure 5: Run 70 (majority cue weights, partner links, no readout speed-up). Only the root is captured cleanly; the probes of levels 1–3 end at $|\cos| \approx 0.4$–$0.5$ with their own node because several of them go to the root or to an ancestor instead. The cues never rotate and the loss floors at 0.14. Section 6 shows that this is sampling noise breaking the symmetry, not a defect of the design.

# 5 How the tree is represented in the trained network

Figure 6 shows run 60 at epoch 1000, and Figure 7 one unit. The representation is explicit:

1. **The first layer is a feature bank of the normals.** Each probe has become $\pm u_j$ for its node (the two probes of a node carry the two poles), with norms $12.3, 9.0, 7.1, 5.5$ by level; each cue has rotated to $\pm u_m$ with norm $\approx 1$. The off-span part of every row has gone. A row of the form $\|w\|\,\mathrm{ReLU}(\pm u_j \cdot x)$ is a half-space detector for one side of one node.
2. **Units are** $\mathrm{ReLU}(\textbf{background} - \textbf{vetoes})$**.** A level-3 unit (node $j$ of level 3, leaf $m$ below it) has weight $\approx -6$ on its own probe and $\approx -5$ on one probe of each of its three ancestors (Figure 7: unit 28 of node 7 vetoes $u_7$, $u_3$, $u_1$, $u_0$), a veto of $-4$ on the opposite-pole cue of its leaf, and small positive weights on the other cues. The vetoes switch the unit off whenever $x$ is on the wrong side of any ancestor, so the unit is active on 3.9 % of the inputs and 96 % of its active inputs lie in a single leaf cell: it is a *leaf-cell detector*, and inside the cell its output follows $\mathrm{sign}(u_m \cdot x)$ through the cues.
3. **The level-3 units carry the function.** Their readouts are $\pm 62$ on average against 4.4 at level 0; removing the 28 units of levels 0–2 raises the test error only from 0.40 to 0.77 %, while removing the 32 level-3 units raises it to 24.8 %. On each leaf cell the logit is a sum of $\mathrm{ReLU}(\pm u_m \cdot x)$ terms with the right signs, which is the ODT itself. The units of levels 0–2 are the *scaffold*: they are the units that pushed the probes of their own node (a level-3 unit has no weight on an ancestor's probe at init, so it cannot create the ancestor's feature, only use it), and they become dispensable once the features exist.

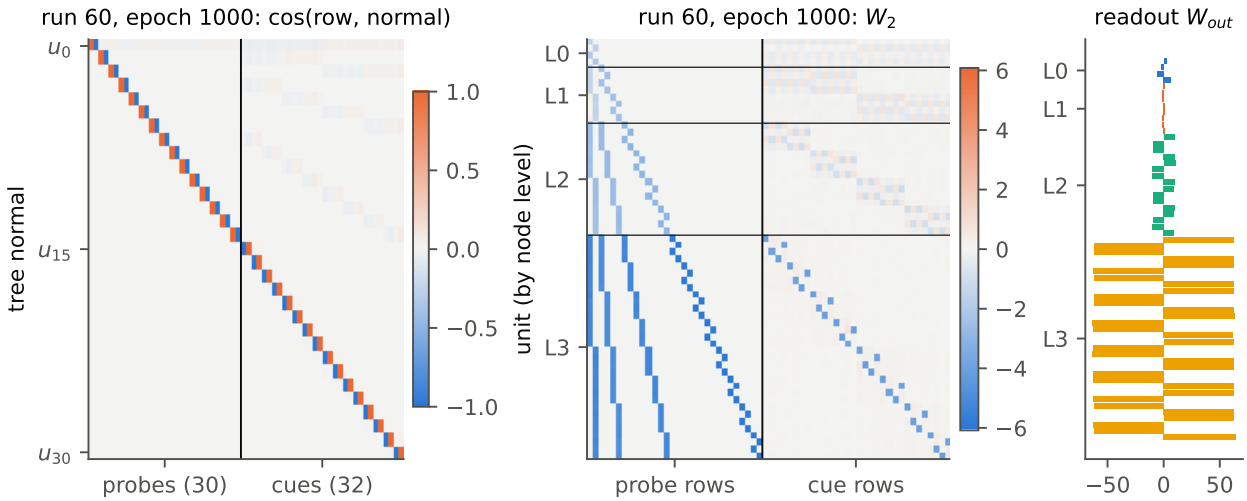



Figure 6: Run 60 at epoch 1000. Left: every probe is $\pm u_j$ and every cue $\pm u_m$. Middle: $W_2$; the level-3 units veto their own probe and the probes of their ancestors (the diagonal bands), and the cue block has become a one-cue-per-unit pattern. Right: readouts by level; the level-3 units dominate.

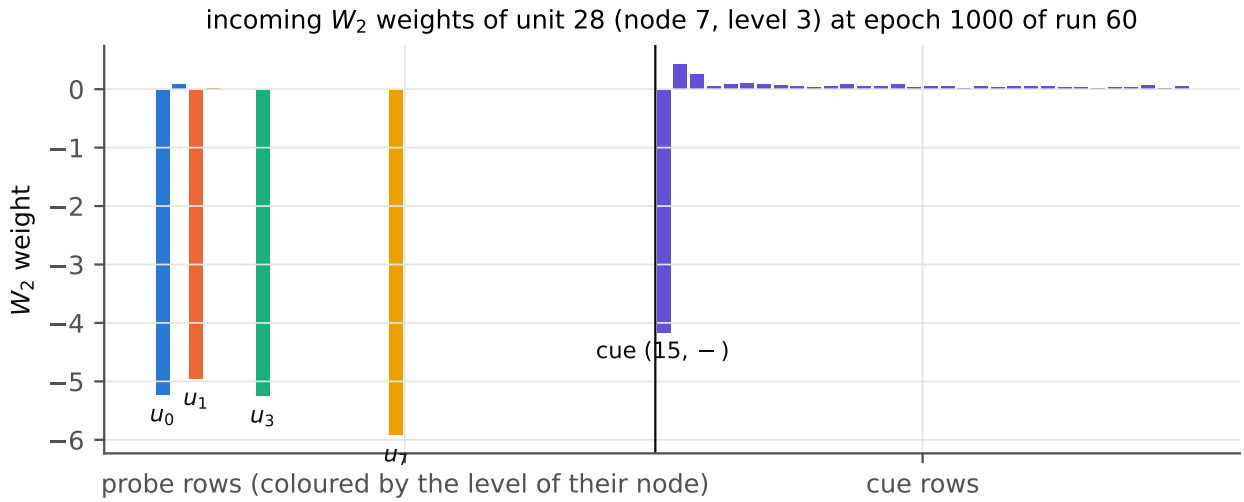



Figure 7: Incoming weights of one level-3 unit (unit 28, node 7) at the end of run 60: vetoes of −5 to −6 on its own probe and on one probe of each ancestor ($u_3$, $u_1$, $u_0$), a veto of −4 on the opposite-pole cue of leaf 15, and small positive weights on the other cues (the background).

## 6 Symmetrised gradients: simulating population gradient flow

**The group.** Let $G$ be generated by the 15 flips $F_j$ (negate $u_j$, exchange the two subtrees below $j$) and the label negation $N$ (negate the 16 leaf normals, which negates $y$); $|G| = 2^{16} = 65\,536$. Each $g \in G$ acts on the parameters by permuting the rows according to their identities (probe $(j, \pm)$, cue $(m, \pm)$), permuting the units according to theirs (node, block sign, path sign), and flipping the signs of the affected coordinates of $W_1$ and of $W_{\text{out}}$. The loss is invariant, $L(g \cdot \theta) = L(\theta)$, and the designed initialisations are fixed points, $g \cdot \theta_0 = \theta_0$. Gradient flow on the population loss therefore stays on the fixed-point set forever; finite-sample SGD leaves it at the first step.

**The symmetrised flow.** We run the same SGD but project every minibatch gradient onto the symmetric subspace: coordinates of $(W_1, W_2, W_{\text{out}})$ are grouped into orbits of $G$ (with signs), each gradient entry is replaced by the signed mean over its orbit, and orbits that contain a coordinate and its own negative are set to zero. This equals the gradient on the minibatch augmented by all 65 536 images of its points, at the cost of one backward pass and a bincount. Of the 7 562 coordinates, 203 orbits remain, 157 of them free (Figure 8); a probe's in-span part, for instance, can only move along its own normal and the normals of its ancestors. Sampling noise survives only inside this subspace, so the trajectory is an estimate of the population flow from the symmetric init, with the usual step-size discretisation.

**Run 74: the flow from run 60's init.** The symmetrised run from run 60's initialisation captures the levels at epochs 9, 15, 21, 30 (SGD: 10, 40, 22, 31), rotates the cues by epoch 801 (SGD: 653) and ends at 0.029 loss and 0.45 % error (Figure 10, left panel of Figure 11). On this init, SGD and the population flow agree: the design's pushes are 30–40 times the sampling noise, so the noise is irrelevant. Run 114 repeats run 74 with the cosine schedule restarted at the unfreeze epoch and reaches 0.006 loss and 0.05 % error; the cold second half of a single cosine was the only thing holding run 74 back.

**Runs 73 and 97: the flow from run 70's init.** Here the two differ. SGD (run 70) captured only the root; the symmetrised flow (run 73, with the frozen phase doubled to 1000 epochs) captures all four levels, at epochs 98, 408, 313, 396, every probe on its own node (Figure 9, right panel of Figure 11). The small side contrast of run 70's weights gives pushes only a few times the sampling noise, and under SGD the probes of levels 1–3 are decided by the sample: they climb to the root or to an ancestor. Under the population flow the same wiring is enough. Run 73's cues did not rotate within its cold free phase; run 97 continues it for 2000 epochs with a fresh cosine and the cues rotate within 23 epochs, ending at 0.010 loss and 0.11 % error with all 31 normals covered. The lesson is that the design's mechanism is deterministic and complete, and that the failures of SGD on the weaker init are symmetry breaking by sampling, not a missing ingredient.

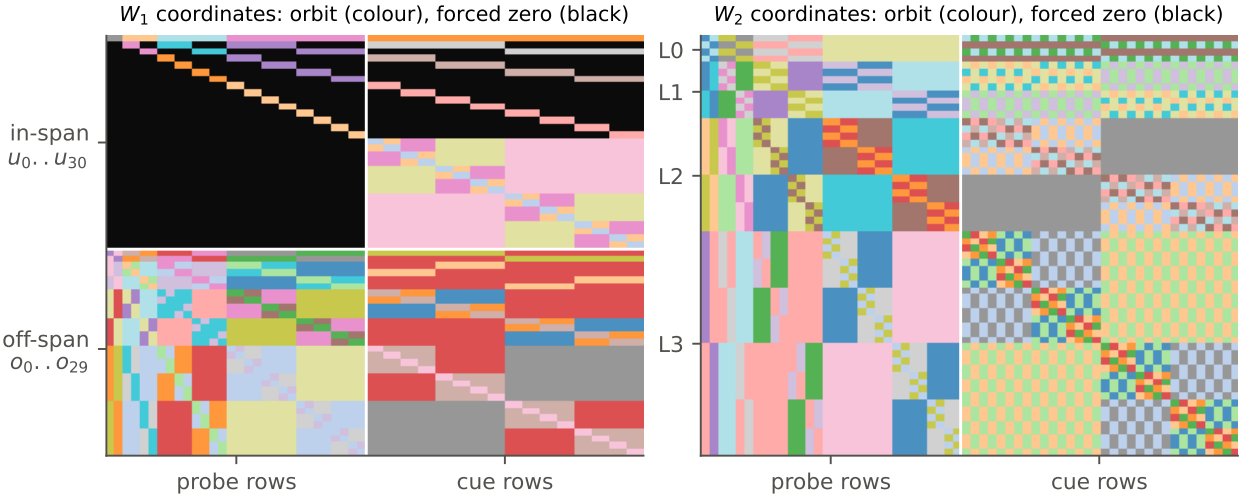



Figure 8: Orbit structure of the parameter coordinates (colour = orbit; colours repeat, black = forced to zero). In $W_1$, the in-span coordinates of a probe are free only on its own normal and its ancestors' (the staircase in the top-left block), the cues' in-span coordinates on the leaf normals, and all off-span coordinates are free. In $W_2$ every block is free, tied across the units of a level.

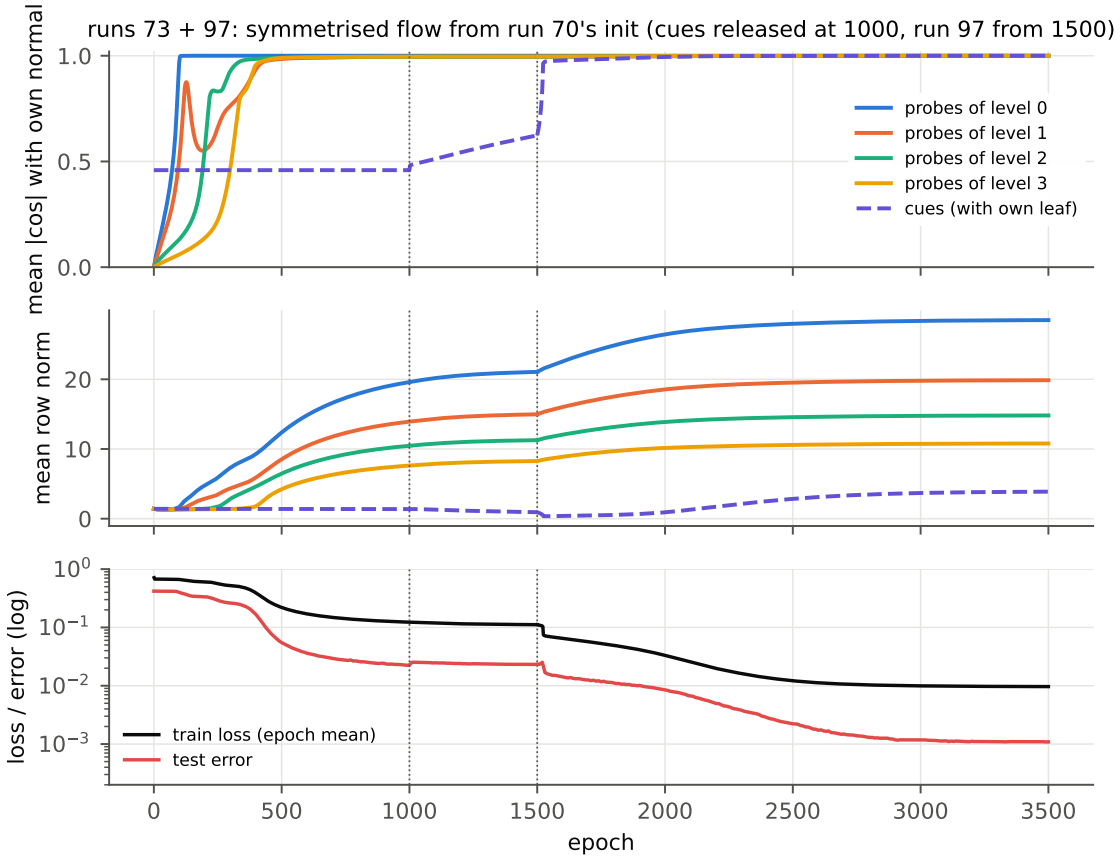



Figure 9: Runs 73 and 97 (symmetrised flow from run 70's init). Every level is captured during the frozen phase (root at 98, levels 1–3 by 408), with the transient dips of levels 1–2 showing the competition between a node and its ancestors that the symmetric flow resolves in favour of the node. The cues rotate only once run 97 restarts the learning rate (epoch 1500); loss and error then fall by an order of magnitude.

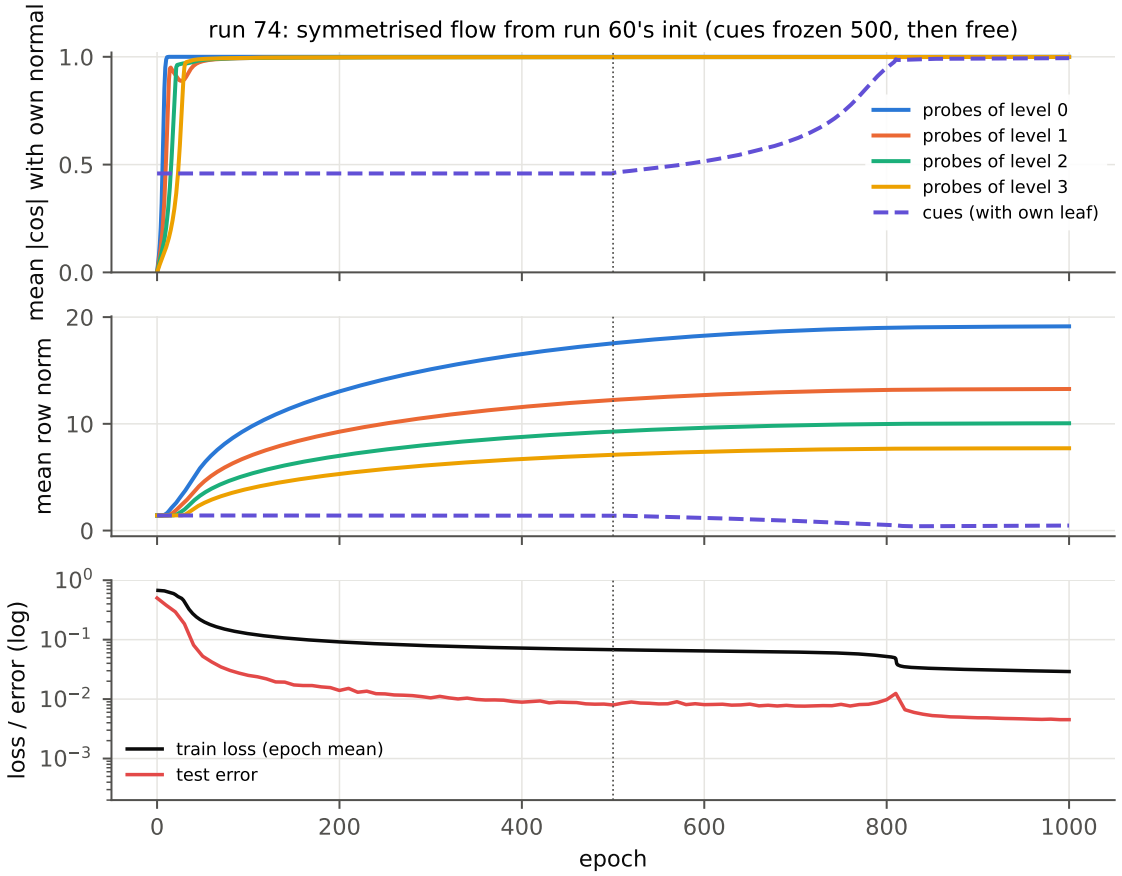



Figure 10: Run 74 (symmetrised flow from run 60's init). Indistinguishable from run 60 at the level of these curves.

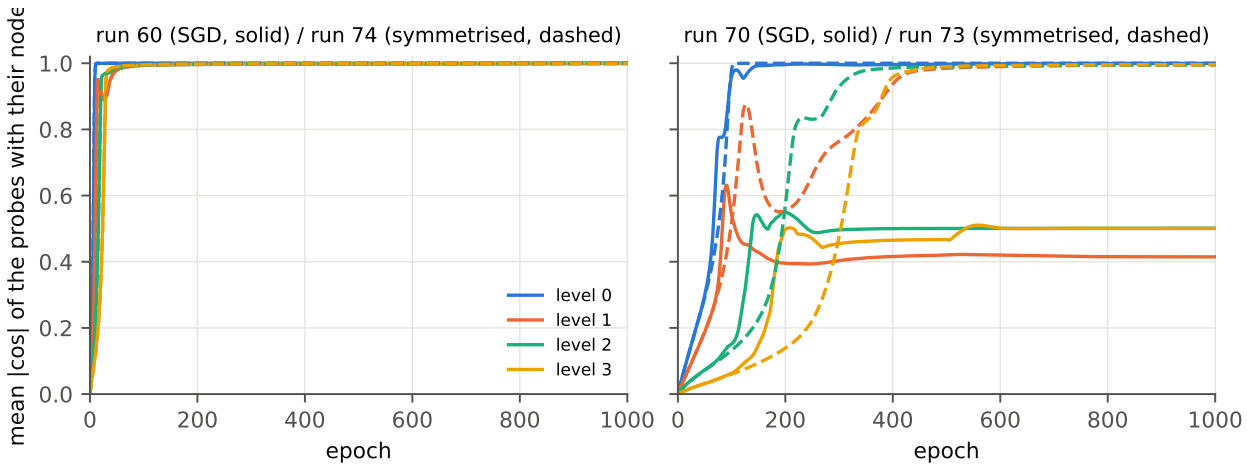



Figure 11: SGD against the symmetrised flow. Left: run 60 and run 74 coincide. Right: run 70 loses levels 1–3 to noise-driven competition, run 73 captures them all.

# 7 What this buys, and where it is going

- **A mechanism, not a correlation.** The lens formula (2) predicts which row moves along which normal from the wiring alone, including the order (root first, because the side contrast of a level-$l$ block sums over $2^{4-l}$ leaves) and the transient ancestor components. The symmetrised runs show the mechanism works without any help from noise.
- **The "cheating" budget is explicit.** The init uses the tree in three places: the cue bias $\beta$, the side contrast, and the absence of off-span components in the cues. Each is now one scalar of the $\alpha$ parametrisation and has been swept (notebook 2, sections 34–38). The flow restores the third by itself within tens of epochs, the first costs time like $1/\beta$ with no threshold in the population flow, and the second is the one quantity gradient descent does not create from a side-symmetric start.
- **The open theoretical question** is the random-init version of this story: a random first layer has a small component of order 0.1 on every routing normal, a random $W_2$ reads the two sides of a node unequally by chance, and the loop "probe component on $u_j \to$ cue side contrast on $j \to$ probe push on $j$" is a per-node instability of the symmetric state whose growth rates should be ordered by level. Making that precise, and testing it with large-batch GD from random inits, is the next line of work.

**Where things are in the repository.** Code: `src/layer_lenses/alpha_init.py` (the $\alpha$ parametrisation of the inits), `symmetric_flow.py` (orbits and projection), `run_analysis.py` and `path_analysis.py` (trajectory and path tools). Runners: `results/designed_init/run_hd2_phased.py` (SGD runs such as 60 and 70) and `run_symmetric_gd.py` (symmetrised runs such as 73, 74, 97, 114). Notebooks: `notebooks/hierarchy_design_runs_2.ipynb` (one section per run, with findings) and `notebooks/two_layer_network_anatomy.ipynb` (choose a run and an epoch; the replay trajectories of runs 60, 70, 73 and 74 are in the repository). Notes: `notes/session_log.md` is the chronological record; this handout and the script that made its figures are in `notes/handout/`.